

Apex Team Building

Final Project IE 6700

Group 1

Francisco Chavez

Harikhumar Prabakaran

617-694-6655

857-396-6394

chavezgonzalez.f@northeastern.edu

prabakaran.h@northeastern.edu

Percentage of Effort Contributed by Student1: 50%

Percentage of Effort Contributed by Student2: 50%

Signature of Student 1: Francisco Chavez

Signature of Student 2: Harikhumar Prabakaran

Submission Date:04/21/2024

Apex Team Building

Apex Legends, developed by Respawn Entertainment, is a high-octane battle royale shooter set in the Titanfall universe. In this game, 20 squads of three players each skydive onto a vast island, with the objective to be the last team standing. Uniquely, players choose from a roster of 'Legends,' each with distinct abilities, roles, and playstyles, offering varied tactical approaches to combat and survival. The game emphasizes team collaboration, strategic positioning, and adaptability to dynamically shrinking play zones. Apex Legends stands out with its emphasis on character-driven gameplay, fast-paced action, and rich storytelling within its evolving universe.

The esports landscape of Apex Legends has rapidly expanded, gaining immense popularity worldwide. It features competitive leagues and high-stakes tournaments that draw the best squads from across the globe. These events highlight advanced levels of gameplay, where teams demonstrate strategic prowess, adaptability, and teamwork in high-pressure scenarios. Our project is focused on creating a software tool designed to help companies build and manage elite professional esports teams for Apex Legends, optimizing their performance for these highly competitive environments.

THEORY

In Apex Legends, each squad comprises three players, who select from a diverse cast of Legends, each with unique abilities and roles such as Offense, Defense, Support, and Recon. The game features various maps, each demanding distinct team strategies and compositions. By leveraging data from Apex Legends leaderboards, which include the top-tier players, our platform simplifies the process of assembling elite teams. It analyzes detailed player statistics, performance metrics, and rankings, enabling teams to identify and recruit players who align with their strategic needs, thus maintaining a competitive advantage in the dynamic realm of esports.

For each Player and Squad, the recruiters will record the following metrics:

- Headshot%
- IG Rank
- Win%
- Damage per game
- Kill/Death

In the Apex Legends esports scene, each team constructs their squads with a core of three primary players, two substitutes, a head coach, and an assistant coach. The selection process emphasizes each player's proficiency with different Legends as well as their performance on various maps and with specific in-game equipment. This strategic approach considers the unique strengths and weaknesses of each player, ensuring a well-rounded and adaptable team ready to face the diverse challenges of competitive play.

While creating the Squads the Team records:

- Squad ID
- Squad Name
- User Name of Head Coach and Assistant Coach

Because esports are gaining more popularity, each team can have zero to multiple sponsorships. These companies are also promoting different tournaments where teams are enrolled. A tournament is a special event where many games are played and where different squads compete. This software also delves into a comprehensive examination of the competitive landscape within Apex Legends esports. It encompasses an in-depth look at rival teams, emerging trends in tournament play, a meta-analysis focused on evolving strategies, and metrics related to fan engagement, alongside a broader market analysis. Then finally the software also outlines the organizational framework supporting the team's operational and competitive endeavors, covering aspects from training schedules to logistical preparations for tournaments.

ENTITY TYPES AND CARDINALITIES

1. Sponsor

A Sponsor can sponsor zero to multiple teams and tournaments. Each Sponsor will have:

- Specific and unique ID
- Name
- Amount of money that is giving
- Region

2. Team

A Team can have zero to multiple Squads. They can participate in multiple Tournaments and can be sponsored by multiple Sponsors. Each Team will have:

- Specific and unique ID
- Name
- Achievements

- Region

3. Tournament

A Tournament can have 1 to multiple Squads and Games. It should have at least one team and can be sponsored by multiple Sponsors. A tournament will hold at least 1 to multiple games. Each Tournament will have:

- Specific and unique ID
- Name
- Region
- Fee
- Reward

4. Squad

A Squad can only belong to one team. A squad is composed by players and coaches, where a player cannot be a coach and a coach cannot be a player. It will have multiple players and coaches. A Squad can be enrolled in 0 to multiple Tournaments. The system will record its metrics too. Each Squad will have:

- Specific and unique ID
- Name

5. Player

A Player is a user that can have just 1 Equipment and Legend. They belong to a squad. The player will have:

- Specific and unique ID
- Country
- Age
- Name
- Role

6. Metrics

Metrics are the data gathered for Players and Squads. Each player and Squad will have just one set of metrics, but the metrics will be gathered for multiple players and squads. Metrics will have:

- Specific and unique ID
- Head Shot%
- IG Rank
- Rank %
- Win %
- Damage per game
- Kill/Death ratio

7. Equipment

Equipment is the set of weapons used by a Player. Equipment will have:

- Specific and unique ID
- Range of weapon

- Type of weapon
- Type of ammunition

8. Legend

Legend is the character used by a Player. Legend will have:

- Specific and unique ID
- Playstyle
- Tactical ability
- Passive ability
- Ultimate ability
- Name

9. Coach

A Coach belongs to a Squad. Each Coach will have:

- Specific and unique ID
- Name
- Career Time
- Role

10. Map

A Map appears in multiple games but a game will have just one specific Map. Each Map will have:

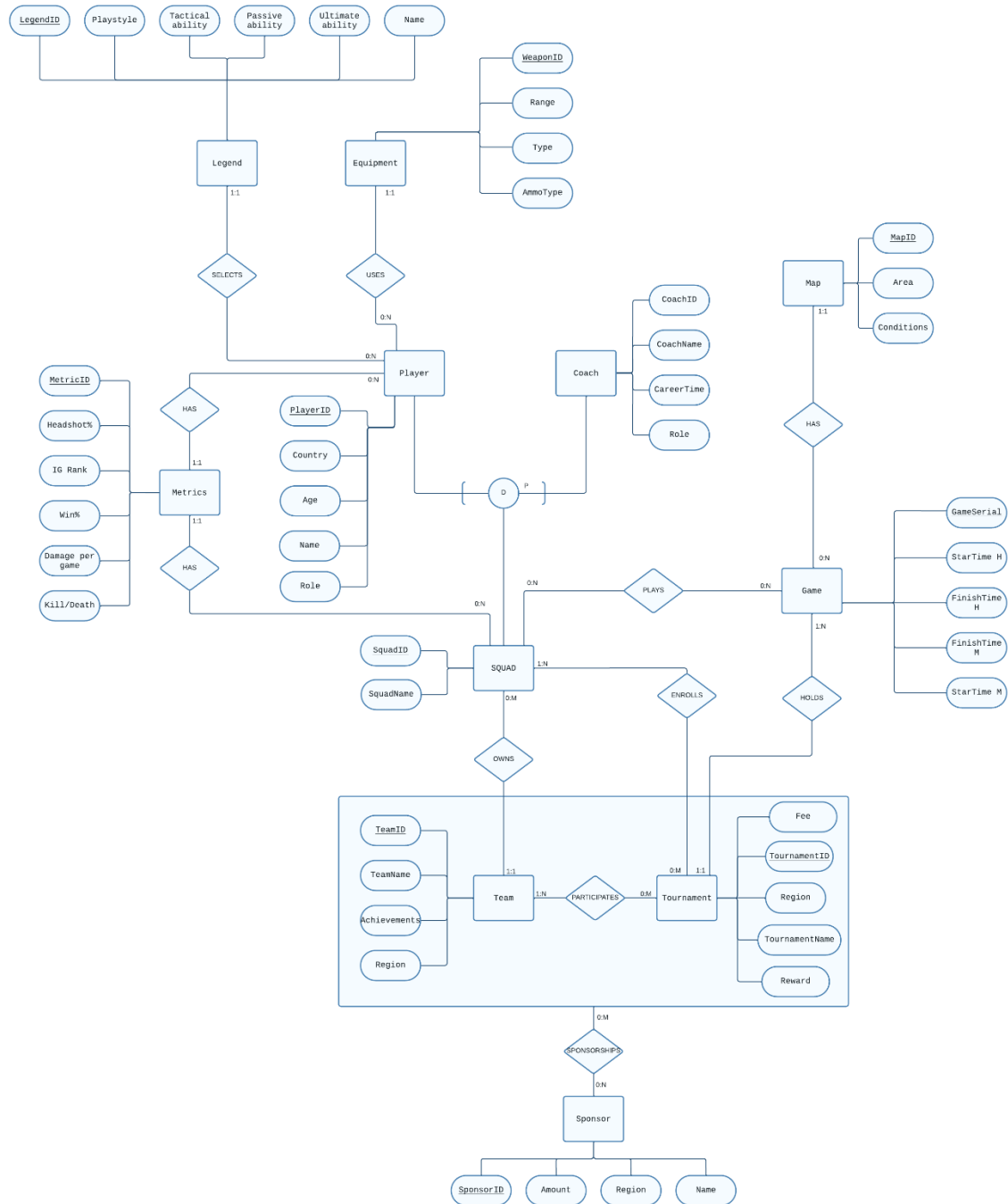
- Specific and unique ID
- Area
- Conditions

11. Game

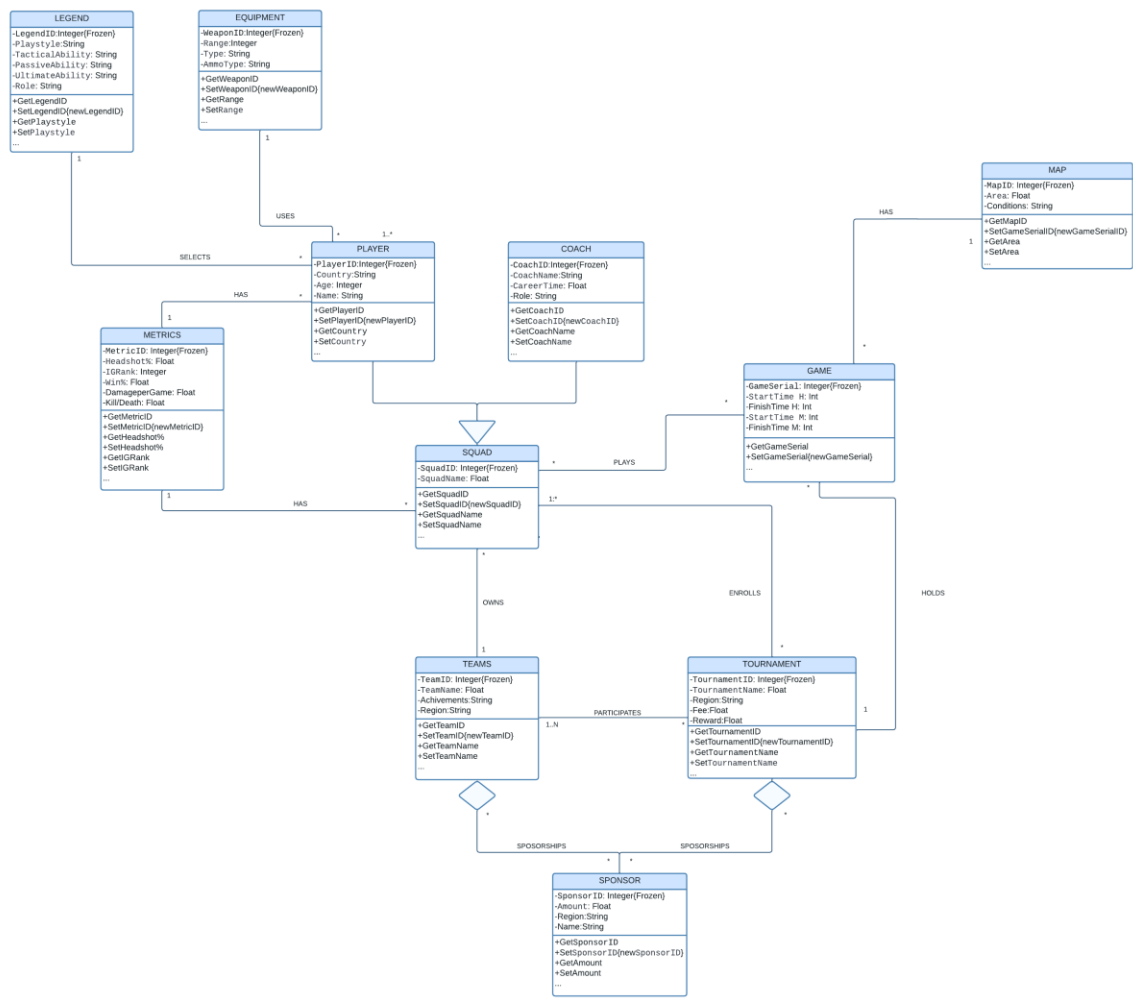
A Game will have multiple squads and can have a map. Multiple games can be part of a tournament. However, each game is unique, therefore it has its ID. Each Game will have:

- Specific and unique Serial
- Start Time
- Finish Time
- Duration

EER DIAGRAM



UML DIAGRAM



RELATIONAL MODEL

The EER model can be mapped to the following relational model (primary keys are underlined; foreign keys are in italics):

Sponsor (SponsorID, Amount, Region, Name)

Team (TeamID, TeamName, Achievements, Region)

Tournament (TournamentID, Fee, Region, TournamentName, Reward)

Sponsorships-Team (*SponsorID*, *TeamID*)

TeamID: Foreign key refers to *TeamID* in relation Team. Null not allowed.

SponsorID: Foreign key refers to *SponsorID* in relation Sponsor. Null not allowed.

Sponsorships-Tournament (*SponsorID*, *TournamentID*)

TournamentID: Foreign key refers to *SquadID* in relation Tournament. Null not allowed.

SponsorID: Foreign key refers to *SponsorID* in relation Sponsor. Null not allowed.

Squad (SquadID, SquadName, *TeamID*)

TeamID: Foreign key refers to *TeamID* in relation Team. Null not allowed.

Enrollment (*SquadID*, *TournamentID*)

SquadID: Foreign key refers to *SquadID* in relation Squad. Null not allowed.

TournamentID: Foreign key refers to *SquadID* in relation Tournament. Null not allowed.

Legend (LegendID, Playstyle, Tactical ability, Passive ability, Ultimate Ability, Name)

Equipment (WeaponID, Range, Type, AmmoType)

Player (*SquadID*, PlayerID, Country, Age, Name, Role, *LegendID*, *WeaponID*)

SquadID: Foreign key refers to *SquadID* in relation Squad. Null not allowed.

LegendID: Foreign key refers to *LegendID* in relation Legend. Null not allowed.

WeaponID: Foreign key refers to *WeaponID* in relation Equipment. Null not allowed.

Coach (*SquadID*, CoachID, CoachName, CareerTime, Role)

SquadID: Foreign key refers to *SquadID* in relation Squad. Null not allowed.

Metrics (MetricID, Headshot%, IG Rank, Win%, Damage per Game, Kill/Death, *PlayerID*, *SquadID*)

SquadID: Foreign key refers to *SquadID* in relation Squad. Null not allowed.

PlayerID: Foreign key refers to *PlayerID* in relation Player. Null not allowed.

Map (MapID, Area, Conditions)

Game (GameSerial, StarTimeH, FinishTimeH, StarTimeM, FinishTimeM, Duration, *MapID*, *TournamentID*)

MapID: Foreign key refers to *MapID* in relation Map. Null not allowed.

TournamentID: Foreign key refers to *TournamentID* in relation Tournament. Null not allowed.

Play (GameSerial, SquadID)

GameSerial: Foreign key refers to *GameSerial* in relation Game. Null not allowed.

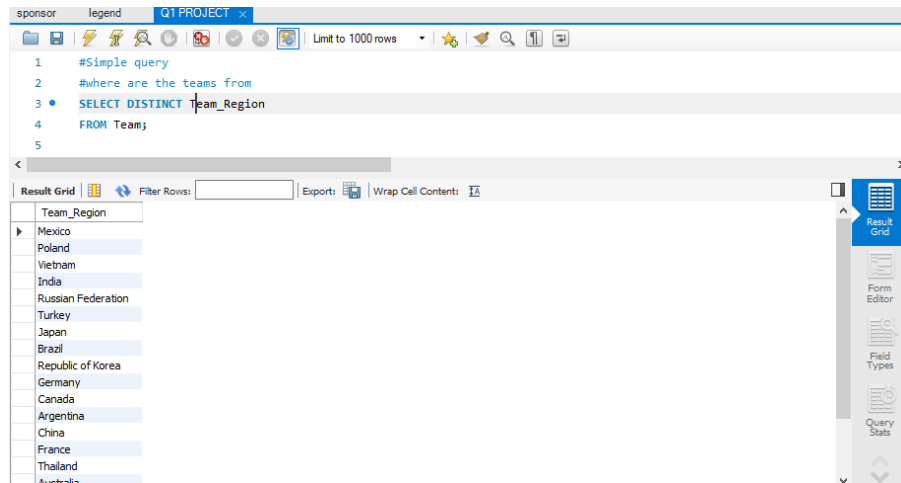
SquadID: Foreign key refers to *SquadID* in relation Squad. Null not allowed.

SQL IMPLEMENTATION

This implementation consists of 16 tables representing the previous relationships presented in our relational model. The population of the data was done with Python since more restrictions and conditions were easily developed.

- 1) These are the regions where all the teams are from
use apex;

```
SELECT DISTINCT Team_Region  
FROM Team;
```



- 2) We want to know how much are teams receiving from the sponsors
use apex;

```
SELECT  
    t.TeamID,  
    t.Team_Name,  
    SUM(s.Amount) AS Total_Sponsorship_Amount  
FROM  
    Team t  
LEFT JOIN  
    Sponsorships_Team st ON t.TeamID = st.TeamID  
LEFT JOIN  
    Sponsor s ON st.SponsorID = s.SponsorID  
GROUP BY  
    t.TeamID, t.Team_Name
```

ORDER BY

Total_Sponsorship_Amount DESC;

The screenshot shows a SQL query editor with the following query:

```
1 #aggregate query
2 #total amount of sponsorship received by each team
3 • SELECT
4     t.TeamID,
5     t.Team_Name,
6     COALESCE(SUM(s.Amount), 0) AS Total_Sponsorship_Amount
7 FROM
8     Team t
9 LEFT JOIN
10    Sponsorships_Team st ON t.TeamID = st.TeamID
```

The result grid below the query shows the following data:

TeamID	Team_Name	Total_Sponsorship_Amount
TM-480	Daniels and Sons	168482.52
TM-854	Church Inc	164370.23
TM-636	Trevino Inc	151335.69
TM-539	Carey-Cole	143792.15
TM-739	Cardenas, Rivas and Lewis	137154.12
TM-944	Carrillo-Schultz	134023.65
TM-478	Maddox-Hernandez	125500.33
TM-389	Anderson-Ray	124446.17
TM-923	Young, Bryant and King	105956.93
TM-668	Barton and Sons	102514.31

- 3) We want to know all the countries where the players are from in each team
use apex;

SELECT DISTINCT t.TeamID, t.Team_Name, p.Country

FROM Team t

INNER JOIN Squad s ON t.TeamID = s.TeamID

INNER JOIN Player p ON s.SquadID = p.SquadID;

The screenshot shows a SQL query editor with the following query:

```
1 # inner join
2 # countries of all players in each team
3 • SELECT DISTINCT t.TeamID, t.Team_Name, p.Country
4 FROM Team t
5 INNER JOIN Squad s ON t.TeamID = s.TeamID
6 INNER JOIN Player p ON s.SquadID = p.SquadID;
```

The result grid below the query shows the following data:

TeamID	Team_Name	Country
TM-103	Scott, Munoz and Griffin	Japan
TM-103	Scott, Munoz and Griffin	France
TM-103	Scott, Munoz and Griffin	Mexico
TM-103	Scott, Munoz and Griffin	Brazil
TM-103	Scott, Munoz and Griffin	China
TM-103	Scott, Munoz and Griffin	Vietnam
TM-103	Scott, Munoz and Griffin	Turkey
TM-103	Scott, Munoz and Griffin	United States
TM-103	Scott, Munoz and Griffin	Indonesia
TM-103	Scott, Munoz and Griffin	Russian Federation

- 4) We want to know the legend name that has been used the most by players with a win value greater than 90

4 FROM Legend
 5 WHERE LegendID = (
 6 SELECT LegendID
 7 FROM Player
 8 WHERE PlayerID IN (
 9 SELECT PlayerID
 10 FROM Metrics
 11 WHERE Win > 90
 12)
 13 GROUP BY LegendID
 14 ORDER BY COUNT(*) DESC
 15 LIMIT 1
 16);
 17

Result Grid

Name
Dr. Ashleigh Reid

```

SELECT Name
FROM Legend
WHERE LegendID = (
  SELECT LegendID
  FROM Player
  WHERE PlayerID IN (
    SELECT PlayerID
    FROM Metrics
    WHERE Win > 90
  )
  GROUP BY LegendID
  ORDER BY COUNT(*) DESC
  LIMIT 1
);

```

- 5) We want to know which equipment has helped players achieve the highest headshot value

The screenshot shows a SQL query editor with a toolbar at the top. The query is a correlated subquery. The result grid below the query shows one row with the values 'WP-004' and 'Assault rifle'. The result grid has columns 'WeaponID' and 'Type_'. The first row has the values 'WP-004' and 'Assault rifle'. The second row has the values 'NULL' and 'NULL'.

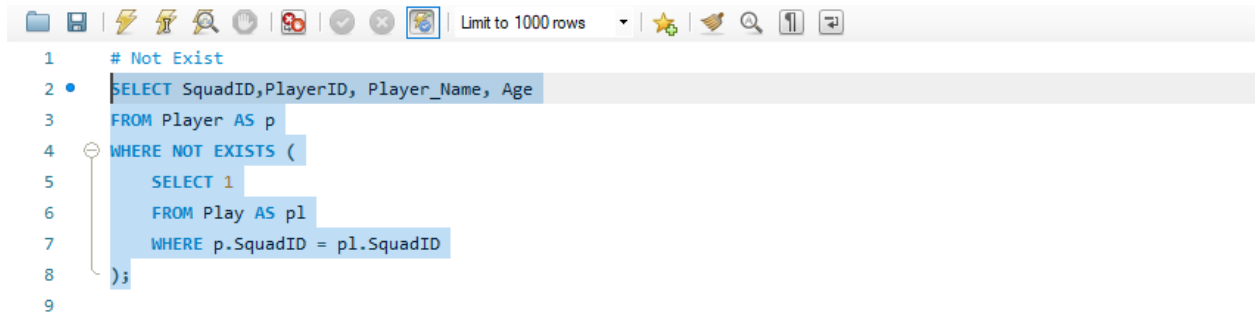
```
1  #correlated query
2  •  SELECT WeaponID ,Type_
3  FROM Equipment AS e
4  WHERE WeaponID = (
5      SELECT WeaponID
6      FROM Player AS p
7      JOIN Metrics AS m ON p.PlayerID = m.PlayerID
8      WHERE m.Headshot = (
9          SELECT MAX(Headshot)
10         FROM Metrics
11     )
12 );
13
14
```

Result Grid

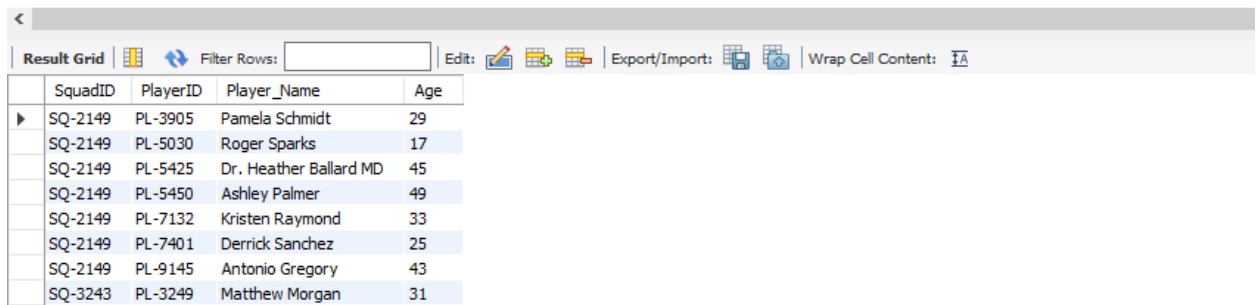
	WeaponID	Type_
▶	WP-004	Assault rifle
*	NULL	NULL

```
SELECT WeaponID ,Type_
FROM Equipment AS e
WHERE WeaponID = (
    SELECT WeaponID
    FROM Player AS p
    JOIN Metrics AS m ON p.PlayerID = m.PlayerID
    WHERE m.Headshot = (
        SELECT MAX(Headshot)
        FROM Metrics
    )
);
```

- 6) We want to know if there are any players who have not participated in any games.



```
1  # Not Exist
2  SELECT SquadID, PlayerID, Player_Name, Age
3  FROM Player AS p
4  WHERE NOT EXISTS (
5      SELECT 1
6      FROM Play AS pl
7      WHERE p.SquadID = pl.SquadID
8  );
9
```



	SquadID	PlayerID	Player_Name	Age
▶	SQ-2149	PL-3905	Pamela Schmidt	29
	SQ-2149	PL-5030	Roger Sparks	17
	SQ-2149	PL-5425	Dr. Heather Ballard MD	45
	SQ-2149	PL-5450	Ashley Palmer	49
	SQ-2149	PL-7132	Kristen Raymond	33
	SQ-2149	PL-7401	Derrick Sanchez	25
	SQ-2149	PL-9145	Antonio Gregory	43
	SQ-3243	PL-3249	Matthew Morgan	31

```
SELECT SquadID, PlayerID, Player_Name, Age
FROM Player AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM Play AS pl
    WHERE p.SquadID = pl.SquadID
);
```

- 7) We want to find out the map that has been used the most in the games

```
1 #query >= ALL
2 SELECT MapID
3 FROM Game
4 GROUP BY MapID
5 HAVING COUNT(*) >= ALL (
6     SELECT COUNT(*)
7     FROM Game
8     GROUP BY MapID
9 );
10
```

MapID
MP-005

```
SELECT MapID
FROM Game
GROUP BY MapID
HAVING COUNT(*) >= ALL (
    SELECT COUNT(*)
    FROM Game
    GROUP BY MapID
);
```

- 8) We want to find the tournament that has had the least number of squads enrolled

```

1  #query <= ALL
2  •  SELECT Tournament_Name, EnrolledSquadsCount, reward
3  FROM (
4      SELECT t.TournamentID, t.Tournament_Name, COUNT(*) AS EnrolledSquadsCount, t.reward
5      FROM Tournament AS t
6      JOIN Enrollment AS e ON t.TournamentID = e.TournamentID
7      GROUP BY t.TournamentID, t.Tournament_Name
8  ) AS TournamentEnrollments
9  WHERE TournamentID = (
10     SELECT TournamentID
11     FROM Enrollment
12     GROUP BY TournamentID
13     HAVING COUNT(*) <= ALL (
14         SELECT COUNT(*)
15         FROM Enrollment
16         GROUP BY TournamentID
17     )
18 );
19

```

Tournament_Name	EnrolledSquadsCount	reward
Burnett-West	10	116601.17

```

SELECT Tournament_Name, EnrolledSquadsCount, reward
FROM (
    SELECT t.TournamentID, t.Tournament_Name, COUNT(*) AS EnrolledSquadsCount,
t.reward
    FROM Tournament AS t
    JOIN Enrollment AS e ON t.TournamentID = e.TournamentID
    GROUP BY t.TournamentID, t.Tournament_Name
) AS TournamentEnrollments
WHERE TournamentID = (
    SELECT TournamentID
    FROM Enrollment
    GROUP BY TournamentID
    HAVING COUNT(*) <= ALL (
        SELECT COUNT(*)

```

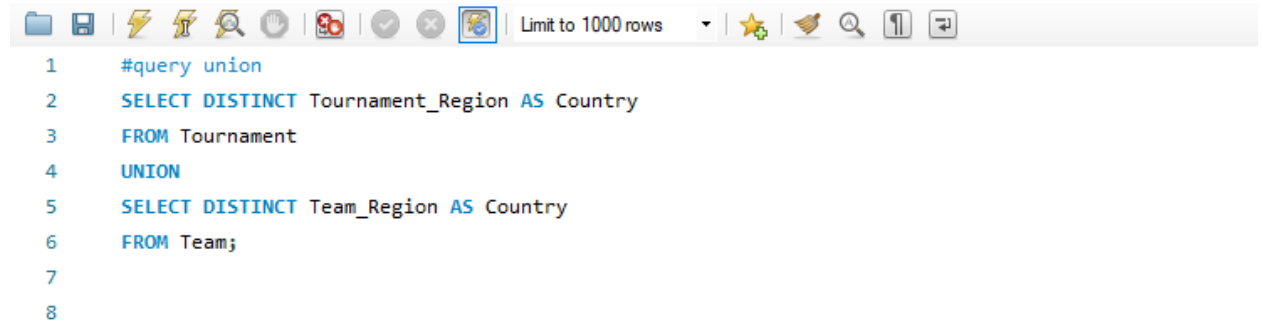


```

FROM Enrollment
GROUP BY TournamentID
)
);

```

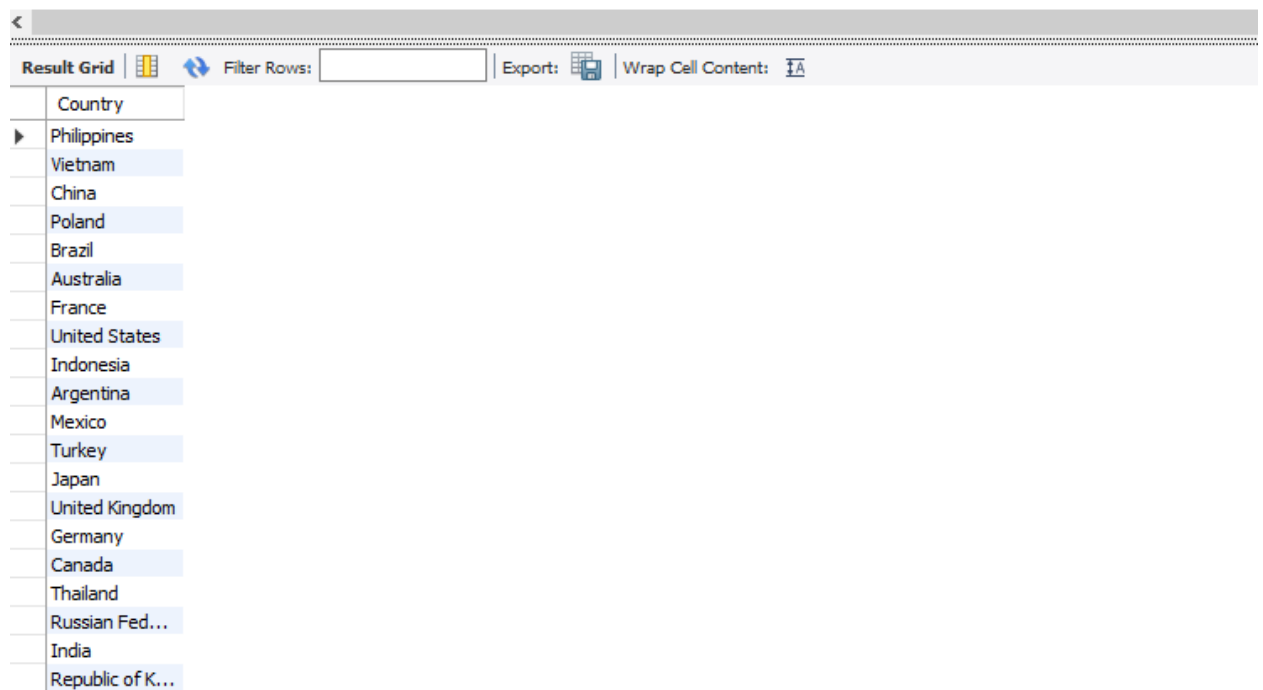
- 9) We want to know the countries that are playing key roles in the esports scene



```

1  #query union
2  SELECT DISTINCT Tournament_Region AS Country
3  FROM Tournament
4  UNION
5  SELECT DISTINCT Team_Region AS Country
6  FROM Team;
7
8

```



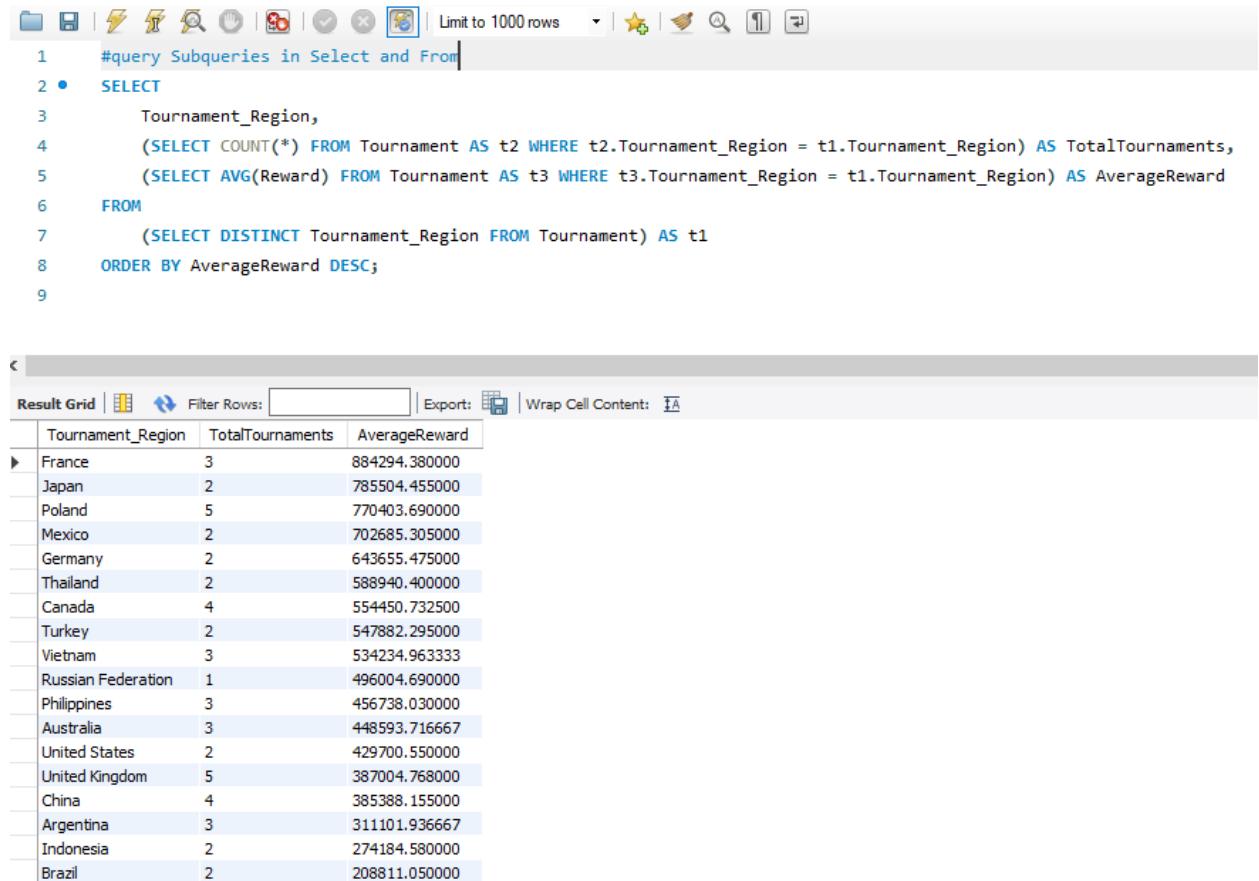
Country
Philippines
Vietnam
China
Poland
Brazil
Australia
France
United States
Indonesia
Argentina
Mexico
Turkey
Japan
United Kingdom
Germany
Canada
Thailand
Russian Fed...
India
Republic of K...

```

SELECT DISTINCT Tournament_Region AS Country
FROM Tournament
UNION
SELECT DISTINCT Team_Region AS Country
FROM Team;

```

10) We want to know the total number of tournaments for each region along with the average reward amount for each region.



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
1 #query Subqueries in Select and From
2 SELECT
3     Tournament_Region,
4     (SELECT COUNT(*) FROM Tournament AS t2 WHERE t2.Tournament_Region = t1.Tournament_Region) AS TotalTournaments,
5     (SELECT AVG(Reward) FROM Tournament AS t3 WHERE t3.Tournament_Region = t1.Tournament_Region) AS AverageReward
6 FROM
7     (SELECT DISTINCT Tournament_Region FROM Tournament) AS t1
8 ORDER BY AverageReward DESC;
9
```

Below the query editor is the 'Result Grid' showing the results of the query. The grid has three columns: 'Tournament_Region', 'TotalTournaments', and 'AverageReward'. The results are ordered by 'AverageReward' in descending order.

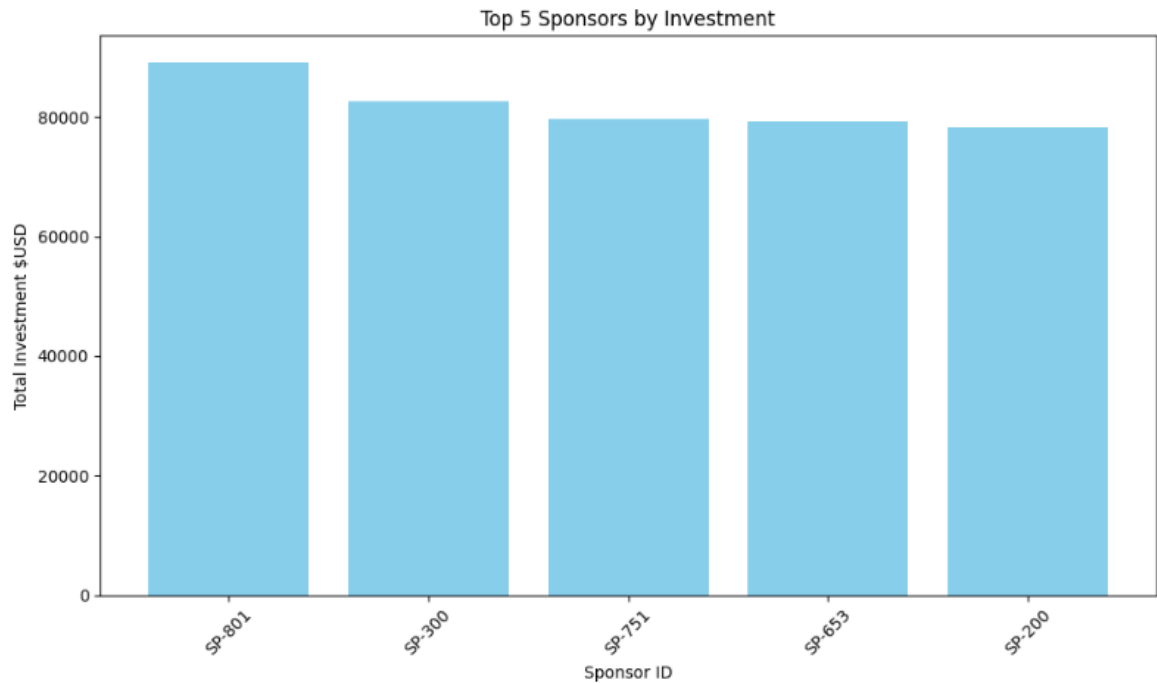
Tournament_Region	TotalTournaments	AverageReward
France	3	884294.380000
Japan	2	785504.455000
Poland	5	770403.690000
Mexico	2	702685.305000
Germany	2	643655.475000
Thailand	2	588940.400000
Canada	4	554450.732500
Turkey	2	547882.295000
Vietnam	3	534234.963333
Russian Federation	1	496004.690000
Philippines	3	456738.030000
Australia	3	448593.716667
United States	2	429700.550000
United Kingdom	5	387004.768000
China	4	385388.155000
Argentina	3	311101.936667
Indonesia	2	274184.580000
Brazil	2	208811.050000

```
SELECT
    Tournament_Region,
    (SELECT COUNT(*) FROM Tournament AS t2 WHERE t2.Tournament_Region =
t1.Tournament_Region) AS TotalTournaments,
    (SELECT AVG(Reward) FROM Tournament AS t3 WHERE t3.Tournament_Region =
t1.Tournament_Region) AS AverageReward
FROM
    (SELECT DISTINCT Tournament_Region FROM Tournament) AS t1
ORDER BY AverageReward DESC;
```

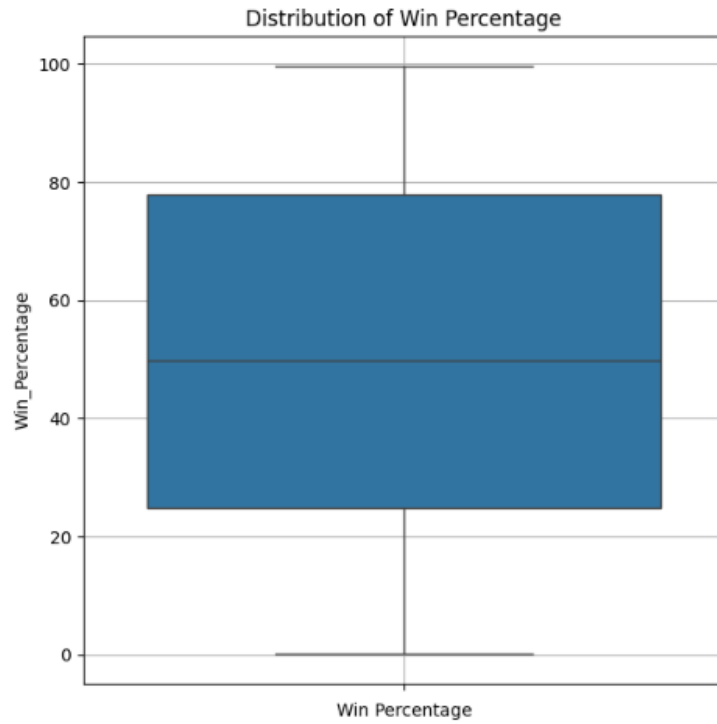
PYTHON IMPLEMENTATION

Using mysql.connector we connect our database to Python to visualize important information for our investors.

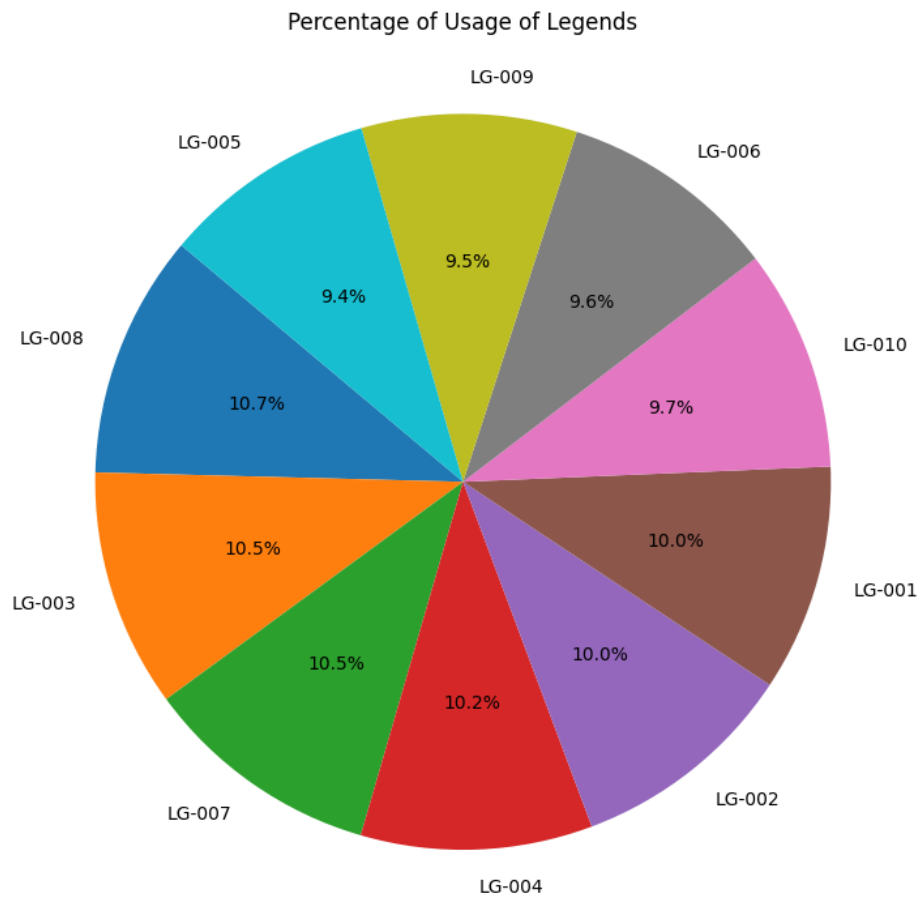
- 1) Amount that in general top investors are willing to invest, as we can see in general if you want to enter to this industry as a sponsor you need to have around \$80,000-\$70,000 USD



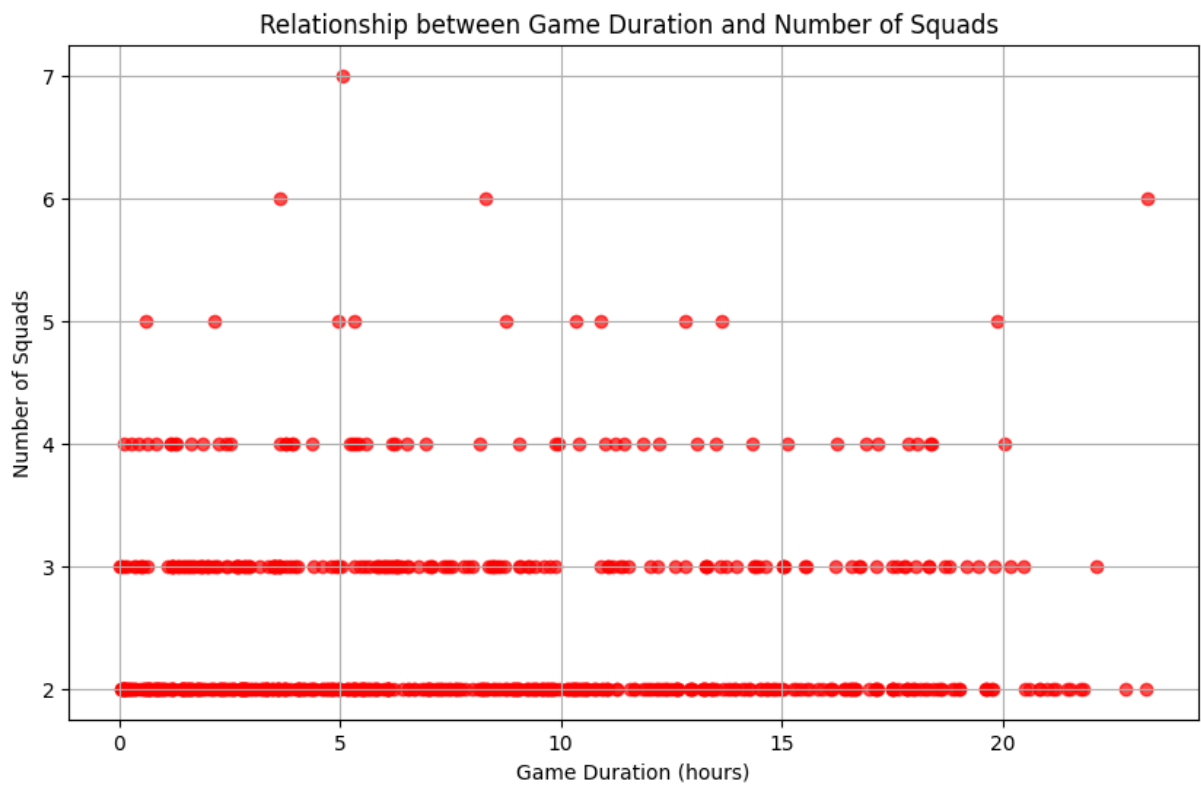
- 2) The distribution of victories per player in general is between 25%-80%, so the players that a team should recruit should be the ones that have this metric above 80%.



3) In general, players are choosing legends that can offer more tactic skills

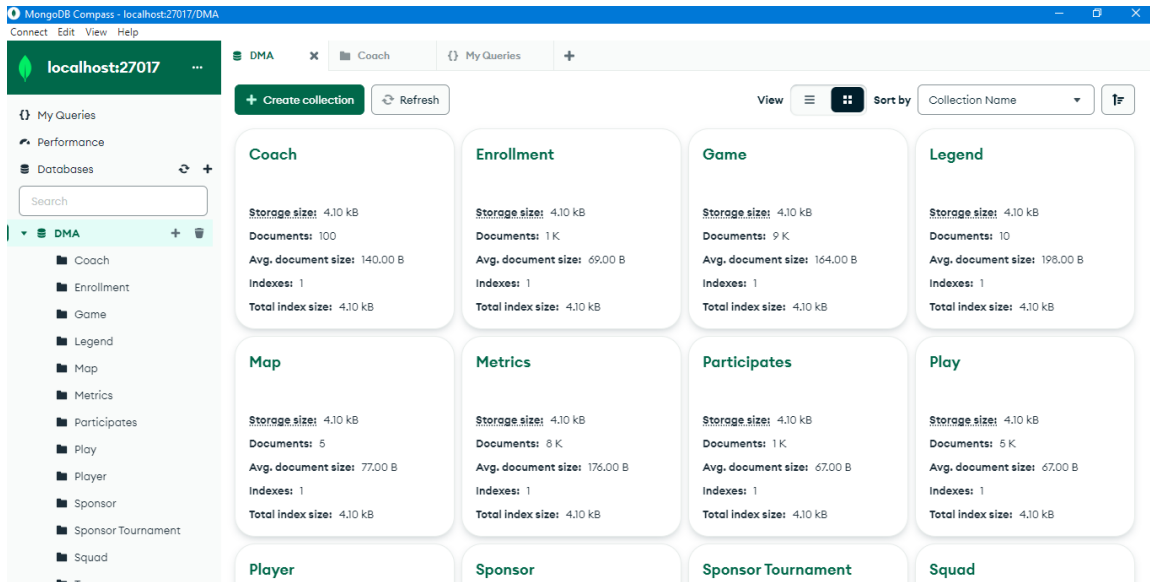


- 4) Relationship between game duration and number of squads. This shows that the more squads the game has, the shorter the duration since the chances of being killed are greater.



NoSQL IMPLEMENTATION

We successfully implemented MongoDB for the NoSQL component of our project. Utilizing collections, we created the database and proceeded to extract valuable insights from the data, showcasing the flexibility and power of MongoDB in handling our dataset.



1) Calculation of the average time per type of coach

```
db.getCollection('Coach').aggregate([
  {
    $group: {
      _id: '$Coach_Role',
      averageCareerTime: {
        $avg: '$Career_Time'
      }
    }
  }
],
{ maxTimeMS: 60000, allowDiskUse: true }
);
```

Results:

```
_id: "Assistant"
averageCareerTime : 5.37037037037037
```

```
_id: "Secondary"
averageCareerTime : 5.055
```

```
_id: "Primary"
averageCareerTime : 4.76969696969695
```

2) Calculation of the average duration per game

```
db.Game.aggregate([
  {
    $addFields: {
      start_time: { $add: [ { $multiply: [ "$Start_time_h", 60 ] }, "$Start_time_m" ] },
      finish_time: { $add: [ { $multiply: [ "$Finish_time_h", 60 ] }, "$Finish_time_m" ] }
    }
  },
  {
    $addFields: {
      duration: {
        $cond: {
          if: { $gte: [ "$finish_time", "$start_time" ] },
          then: { $subtract: [ "$finish_time", "$start_time" ] },
          else: { $add: [ { $subtract: [ 24*60, "$start_time" ] }, "$finish_time" ] }
        }
      }
    }
  },
],
```

```

{
  $group: {
    _id: null,
    totalDuration: { $sum: "$duration" },
    count: { $sum: 1 }
  }
},
{
  $project: {
    _id: 0,
    averageDuration: { $round: [{ $divide: ["$totalDuration", "$count"] }, 2] }
  }
}
])

```

Results:

averageDuration : 719.28

3) Calculation of the win% per squad

```

db.getCollection('Metrics').aggregate(
[
  {
    $group: {
      _id: '$SquadID',
      avgWin: { $avg: '$Win' }
    }
  }
],
{ maxTimeMS: 60000, allowDiskUse: true }
);

```


Results:

_id: "SQ-7476" avgWin : 51.894999999999996
_id: "SQ-6171" avgWin : 57.274
_id: "SQ-8826" avgWin : 45.358
_id: "SQ-1749" avgWin : 45.178
_id: "SQ-1698" avgWin : 70.28
_id: "SQ-1310" avgWin : 48.6875
_id: "SQ-8738" avgWin : 64.11454545454545
_id: "SQ-9423" avgWin : 21.4625
_id: "SQ-1290" avgWin : 47.413333333333334
_id: "SQ-5736" avgWin : 34.567142857142855

Conclusions and Recommendations

The Apex System is a Database Management system that companies can utilize to construct and oversee elite professional esports teams specifically tailored for the game Apex Legends, thereby optimizing their performance in highly competitive environments. This system is capable of recording essential features such as sponsors, tournaments, squads, players, legends, maps, metrics, and more. Through implementation in various environments such as SQL, Python, and MongoDB, along with appropriate data, the system can provide vital insights and graphical representations to facilitate data-driven decision-making.

The current solution has enabled us to identify key countries in the esports scene and discern the best players to recruit for teams. Additionally, we have uncovered various relationships between game duration and the composition of squads participating. This system is easily scalable and adaptable, as it can accommodate multiple battle royale games and serve as a foundation for other types of esports. For future enhancements, the inclusion of more data could illuminate additional trends and relationships that were not explored in this project.

Finally, we believe that the implementation of this system will lead to cost reductions and improved investments for all stakeholders, ultimately enhancing the competitiveness and popularity of esports as a sport.